UML Class Diagrams

Unified Modeling Language

- UML is a way to model your software.
- Take client requirements and produce a UML Diagram
  - This would then be used for implementation in ANY programming language.
- UML is language-independent/language-agnostic.
  - Based on the UML, you can write your entire code (in any language).

Unified Modeling Language

- Essentially different types of flowcharts.
- Various types of UML so that you can observe your software from different perspectives.
- We will focus on Class Diagrams in this course.
  - Also required for your project.
  - Shows the relationships between those classes.

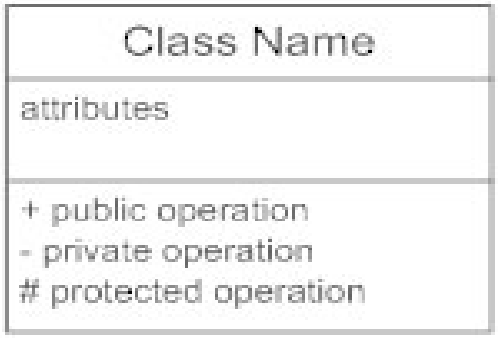
Unified Modeling Language

- Class is represented by a rectangle/block, and generally split into three sections,
  - class name at the top,
  - attributes in the middle and,
  - methods at the bottom.

- The attributes/methods are optional.
  - You must show them in your final submission though.

Unified Modeling Language

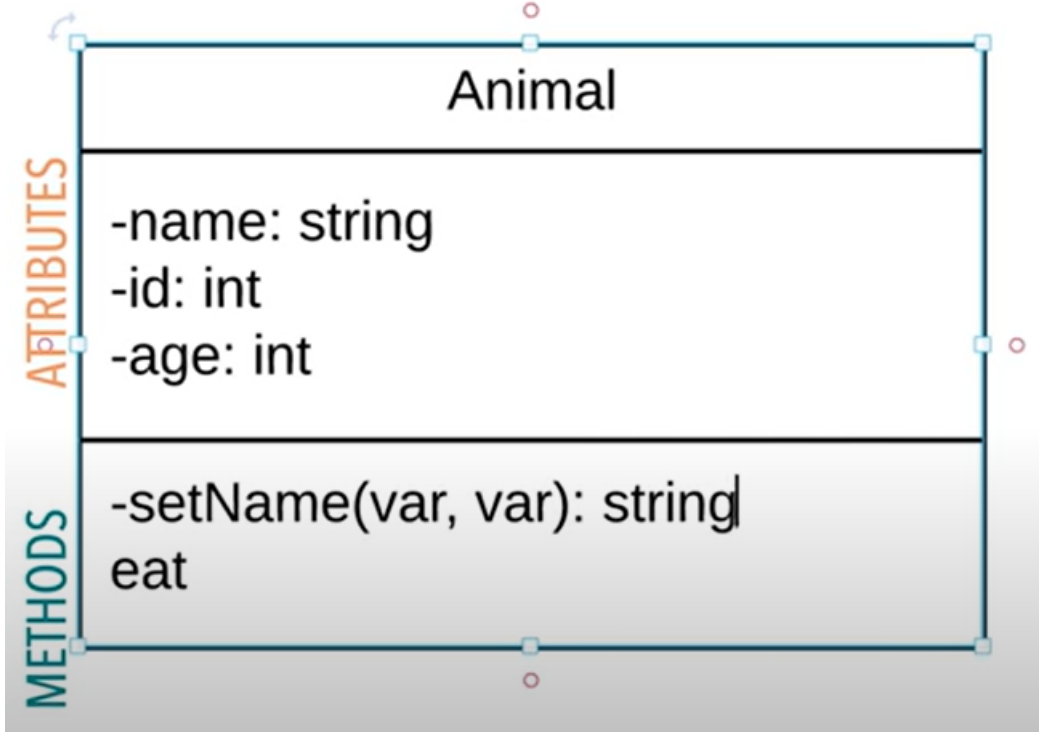
- Access Specifiers for **each member (attributes and methods)** are also shown on a UML Class Diagram.
  - **`public, +`**
  - **`private, -`**
  - **`protected, #`**



This is what the class would look like. Use the formatting for the attributes as shown *(note that the negative signs prefixing the names of the attributes are indicating the **visibility**, private, in this case)*.

For the methods, use the format,
`(visibility)methodName(dtype1, dtype2, …): returnType`

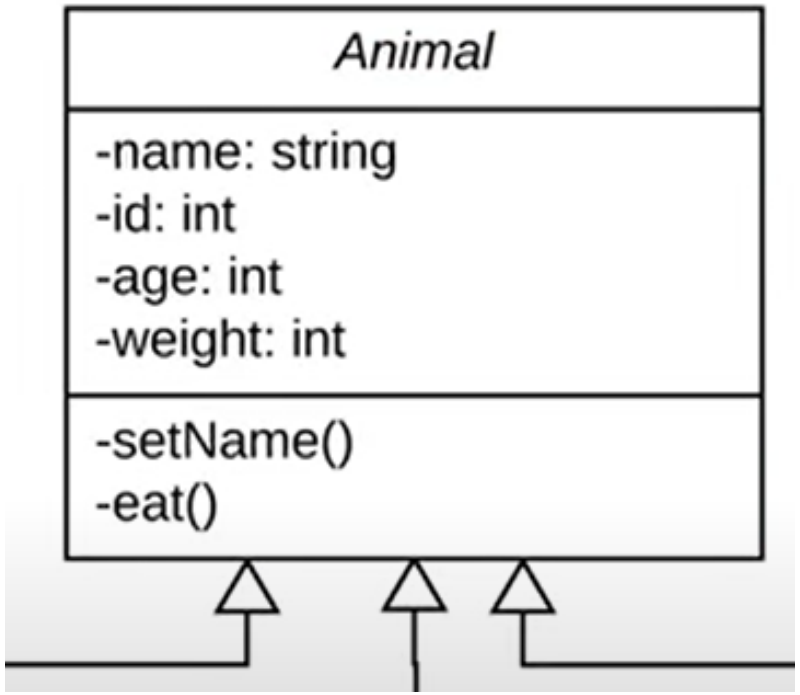
For example,
`+setName(string): void`

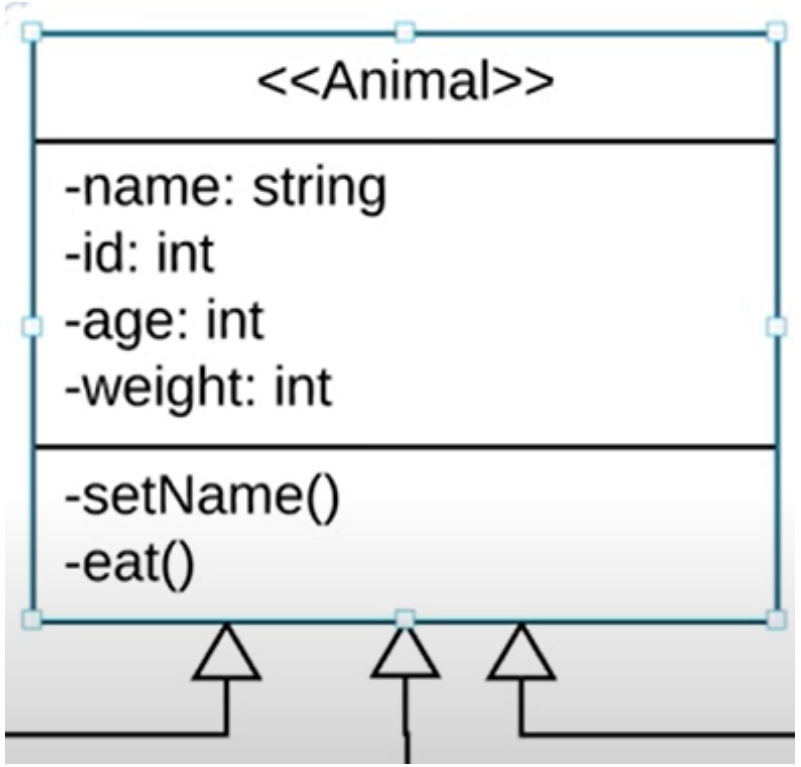


Watch the linked video (~10 mins), it gives a good summary of all the various concepts related to the content covered in this presentation.

https://www.youtube.com/watch?v=UI6lqHOVHic

When working with **abstract classes**, keep their names italicized or within two angled brackets (not italicized).





https://www.youtube.com/watch?v=UI6lqHOVHic

Multiplicity (or Cardinality) in UML

Multiplicity (or Cardinality) in UML

- https://www.umlboard.com/docs/relations/multiplicity/
  - Following explanation is inspired from the above link.

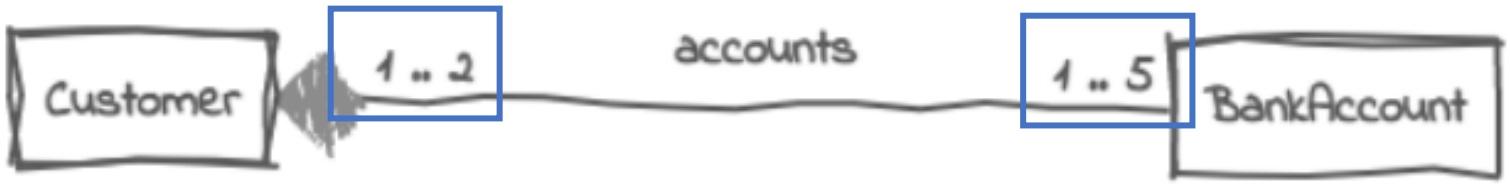
- In UML, **multiplicity** describes how many instances of one class can be connected to an instance of another class through a given relationship *(relationships are coming soon)*.

- *Some examples.*

The following is how multiplicity is represented in UML Class Diagrams.

Let’s see how this will be read.

Keep in mind the statement, *“… **multiplicity** describes how many instances of one class can be connected to an instance of another class …”*.



- Consider the diagram as **two parts** so that you only see the multiplicity close to any one class.
- Assume that there is a '1' multiplicity close to the other class.

- Read, "*One* **`Customer`** *has between one and five* **`BankAccounts`**".

- The **"1..5"** means that there can be between one and five (both inclusive) **`BankAccounts`** for one **`Customer`**.

Keep in mind, *"… **multiplicity** describes how many instances of one class can be connected to an instance of another class …"*.

- This is the **second part**, where you see the multiplicity close to the other class.
- Now, assume that there is a ‘1’ multiplicity close to the **`BankAccount`** class..

- Read, “*One* **`BankAccount`** *belongs to between one and two* **`Customers`**”.

- The **“1..2”** means that there can be between one and two (both inclusive) **`Customers`** for one **`BankAccount`**.

Keep in mind, *“… **multiplicity** describes how many instances of one class can be connected to an instance of another class …”*.

- Now return back to the original diagram.
- You need to combine both parts into a single statement.

- Read, "*One* **`Customer`** *has one to five* **`BankAccounts`** ***AND*** *one* **`BankAccount`** *belongs to one or two* **`Customers`**".

- So every time you see multiplicities on a UML Class Diagram, break it down into two parts.

Keep in mind, *"… **multiplicity** describes how many instances of one class can be connected to an instance of another class …"*.

- This is a table that shows **some common multiplicity symbols** that you may come across, or may require in your project.

- *We have already see the highlighted ones.*

TABLE 10.2 The UML Multiplicity Symbols

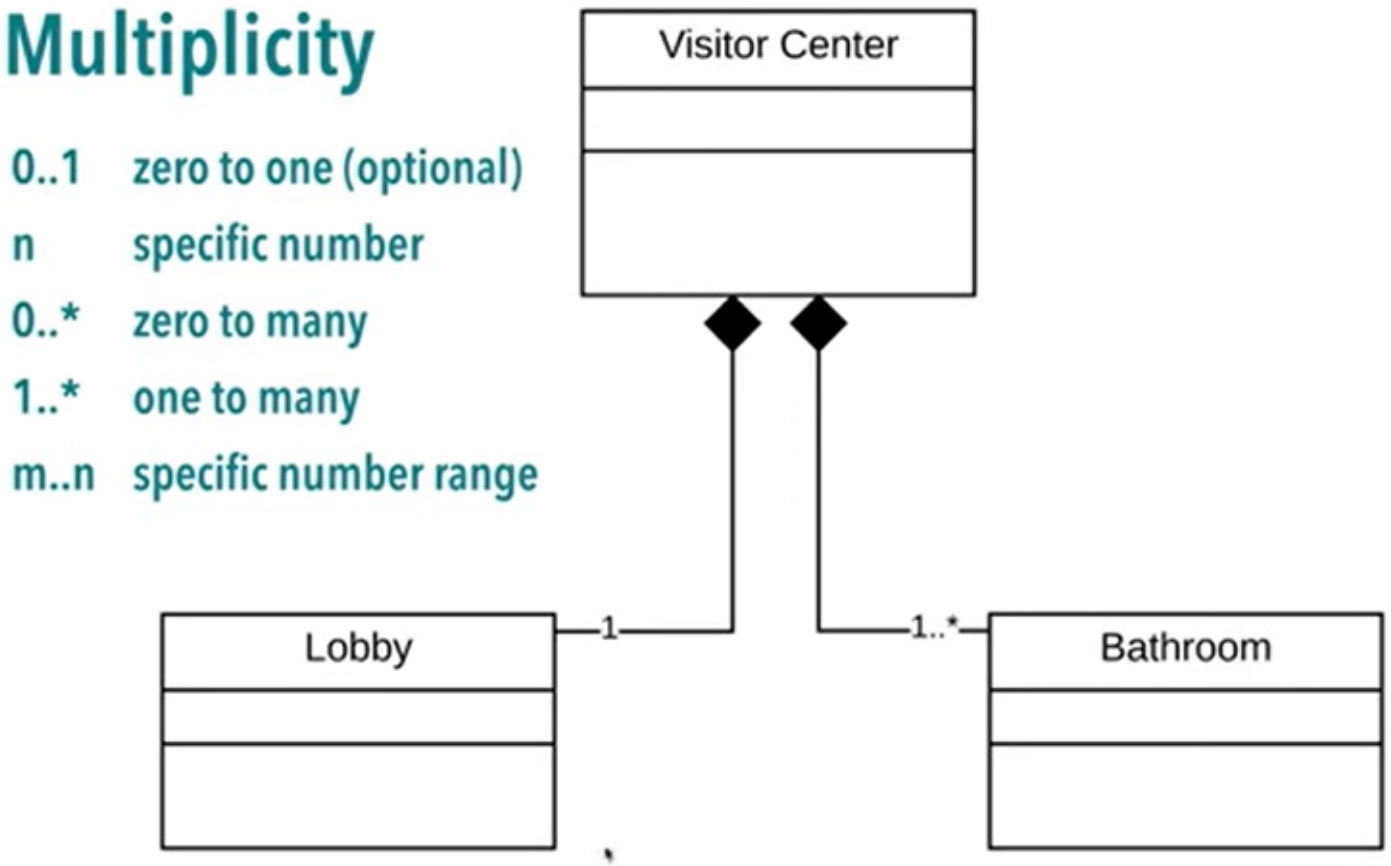
Symbol	*Meaning*
1	One
*	Some (0 to infinity)
0..1	None or one

TABLE 10.2 Continued

Symbol	*Meaning*
1..*	One or more
2..4	Two, three, or four
7,11	Seven or eleven

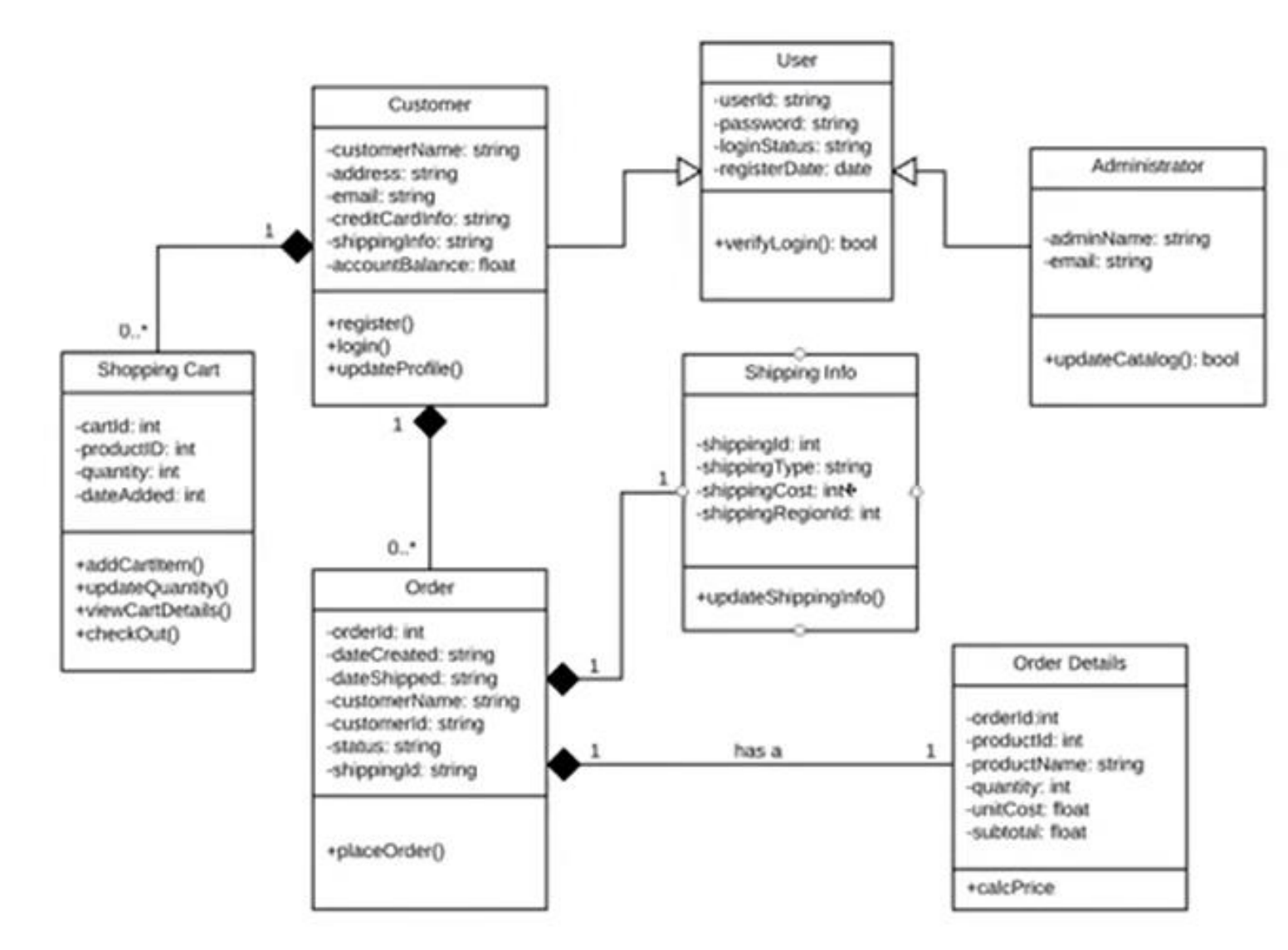
Robert Lafore, page 489, "Multiplicity in the UML"

The video referred to earlier also briefly talks about multiplicity as well.



https://www.youtube.com/watch?v=UI6lqHOVHic

(SS) Check out a full-fledged example discussed in the video.



https://www.youtube.com/watch?v=UI6lqHOVHic

Object Relationships

Object Relationships

- Programming is full of recurring patterns, relationships and hierarchies.
- There are some relationships that you will see often in the context of OOP.
- It’s a good idea to formally be aware about these relationships so that you can write “good” code.
- We have seen **inheritance** already: **“is-a”** relationship.
- Let’s see **Object Composition**.

Object Composition

- Object composition models a **“has-a”** relationship.
- A car “has-a” transmission. Your computer “has-a” CPU. You “have-a” heart.
- By this definition, you can also think of classes and structs as compositions since they have data members (attributes) of various data types.

Object Composition

- There are two subtypes of object composition.
  - Composition
  - Aggregation

- Note on terminology for the purposes of this lecture:
  - *Object composition* would refer to both subtypes (above).
  - *Composition* would be the actual subtype.

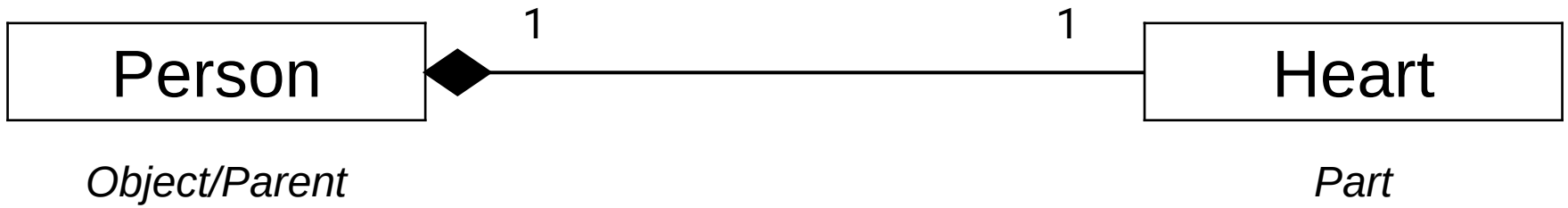
Composition

https://www.learncpp.com/cpp-tutorial/composition/

Composition

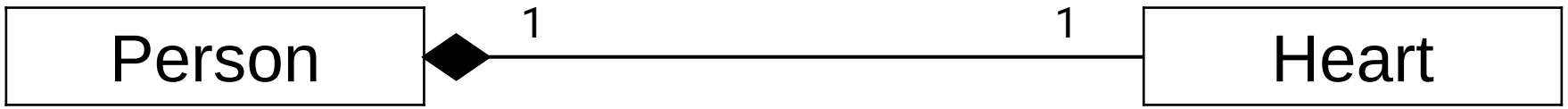
- A good real-life example of a composition is the relationship between a person's body and a heart.

- The Heart (*or part*) is "part-of" the Person (*or object/parent*).



Composition

- Notice the following.
  - The part (heart) is-part of the object (person).
  - The part (heart) can belong to one object (person) at a time.

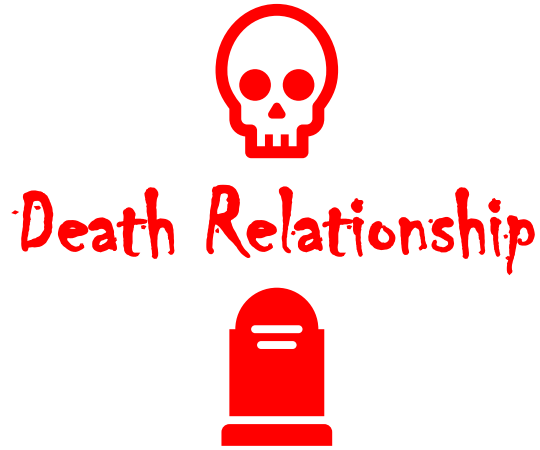


- The part (*the two integers*) is-part of the object (*Fraction-type object*).

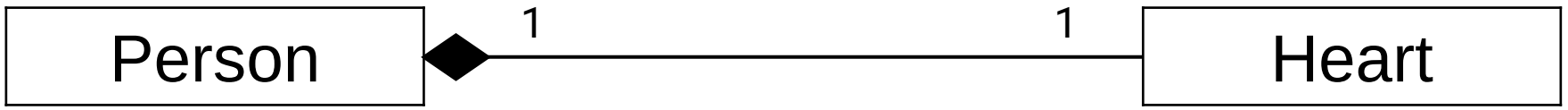
- The part (*the two integers*) can belong to one object (*Fraction-type object*) at a time.
  - It cannot be the case that the integers of one Fraction object are being used elsewhere (e.g. in another Fraction object).

```
1  class Fraction
2  {
3  private:
4      int m_numerator;
5      int m_denominator;
6
7  public:
8      Fraction(int numerator=0, int denominator=1)
9          : m_numerator{ numerator }, m_denominator{ denominator }
10     {
11     }
12 };
```

Composition

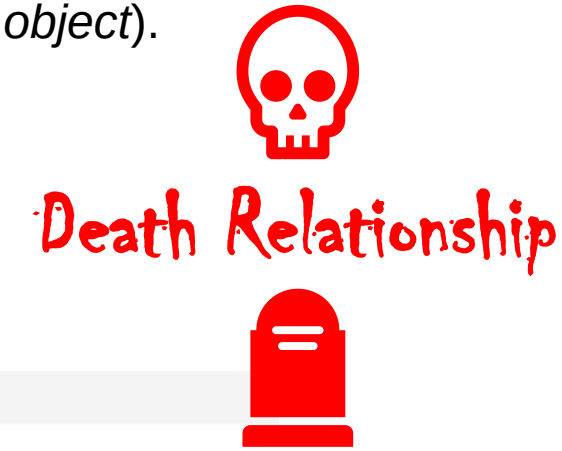


- Notice the following.
  - The part (heart) is-part of the object (person).
  - The part (heart) can belong to one object (person) at a time.
  - The part (heart) has its existence managed by the object (person).



- The part (*the two integers*) has its existence managed by the object (*Fraction-type object*).

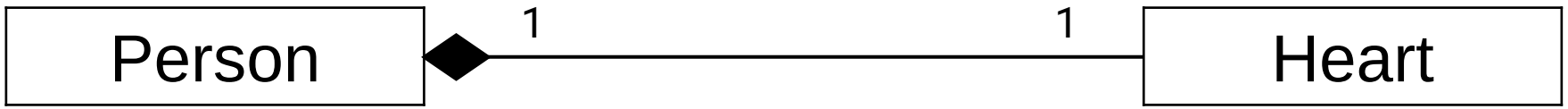
- It cannot be the case that the numerator and denominator,
  - Are brought into existence before the Fraction object is created.
  - Are destroyed before the Fraction object is destroyed.




```
1  class Fraction
2  {
3  private:
4      int m_numerator;
5      int m_denominator;
6
7  public:
8      Fraction(int numerator=0, int denominator=1)
9          : m_numerator{ numerator }, m_denominator{ denominator }
10     {
11     }
12 };
```

Composition

- Notice the following.
  - The part (heart) is-part of the object (person).
  - The part (heart) can belong to one object (person) at a time.
  - The part (heart) has its existence managed by the object (person).
  - The part (heart) does not know about the existence of the object (person).



- The part (*the two integers*) does not know about the existence of the object (*Fraction-type object*).

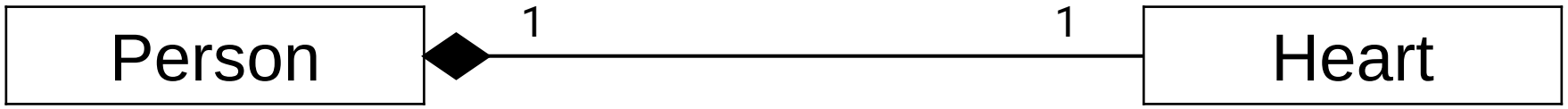
- It cannot be the case that the numerator and denominator, somehow, have an idea that they are part of an object.
  - It's a unidirectional relationship.

```
1  class Fraction
2  {
3  private:
4      int m_numerator;
5      int m_denominator;
6
7  public:
8      Fraction(int numerator=0, int denominator=1)
9          : m_numerator{ numerator }, m_denominator{ denominator }
10     {
11     }
12 };
```

“part-of” instead of “has-a”

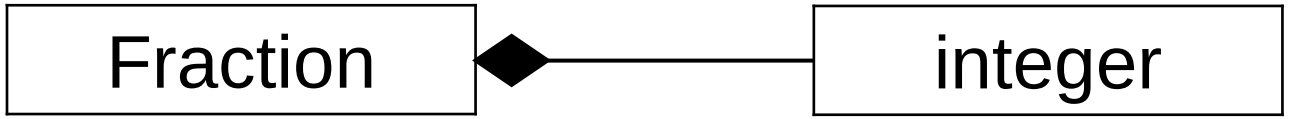
- That’s why the part in a composition relationship is part-of the object.
  - It is more precise/strict than simply saying that a person has-a heart.

- Heart **is-part** of a Person.
- Numerator and Denominator **are-part** of a Fraction.



How is composition implemented?

- A struct or a class with attributes is considered a composition class.
  - However, in UML Class Diagram, we **will not** represent something like this.



- We usually show relationships in UML where **classes** get involved.

How is composition implemented?

- Compositions that need to do **dynamic allocation or deallocation** may be implemented using **pointer data members**.

- In this case, the composition class should be responsible for doing all necessary memory management **itself** (not the user of the class).

- *Example on the next slide.*

- I claim that this class is a composition – the parts are **an integer**, and **an integer array** (we have a pointer member for that).

- Let's see if the class, **`MyList`**, fulfils all the requirements of being a composition.

- Notice the following.
  - The part (heart) is-part of the object (person). ✓
  - The part (heart) can belong to one object (person) at a time. ✓
  - The part (heart) has its existence managed by the object (person). ✓
  - The part (heart) does not know about the existence of the object (person). ✓

It is indeed a composition!

- *The integer and the integer array are surely contained within the **`MyList`**-type objects.*

- *The integer and the integer array are different entities for different objects.*

- *The integer and the integer array come alive and die with objects of this class (for the array, take note of the ctor and dtor).*

- *You can access the integer and the array using objects of this class, but you cannot (somehow) access the object using the attributes.*

```
class MyList
{
public:
    MyList(int size)
        : m_size{ size }, m_data{ new int[size] }
    {}

    ~MyList()
    {
        delete m_data;
        m_data = nullptr;
    }

    MyList(const MyList& ob) = delete;

    MyList& operator= (const MyList& ob) = delete;

private:
    int m_size;
    int* m_data;
};
```

Question: What if I allowed for shallow copies in this class? Would this still be a composition?

No! *Multiple objects would be sharing the same array.*
This is ***aggregation*** *instead.*

- Notice the following.
  - The part (heart) is-part of the object (person). ✓
  - The part (heart) can belong to one object (person) at a time. ✓
  - The part (heart) has its existence managed by the object (person). ✓
  - The part (heart) does not know about the existence of the object (person). ✓

It is indeed a composition!

- *The integer and the integer array are surely contained within the* ***`MyList`****-type objects.*
- *The integer and the integer array are different entities for different objects.*
- *The integer and the integer array come alive and die with objects of this class (for the array, take note of the ctor and dtor).*
- *You can access the integer and the array using objects of this class, but you cannot (somehow) access the object using the attributes.*

```
class MyList
{
public:
    MyList(int size)
        : m_size{ size }, m_data{ new int[size] }
    {}

    ~MyList()
    {
        delete m_data;
        m_data = nullptr;
    }

    MyList(const MyList& ob) = delete;

    MyList& operator= (const MyList& ob) = delete;

private:
    int m_size;
    int* m_data;
};
```

Another example in the reading

- Self-study.

- There is another example in the associated reading which may help you understand the concept further.

Bending rules…

- There will be situations where the four conditions that we saw are bended to varying extents *(read the associated reading)*.

- ***Key point:*** The composition class should manage the lifetimes of its parts **without** the user of the composition needing to manage anything.

Aggregation

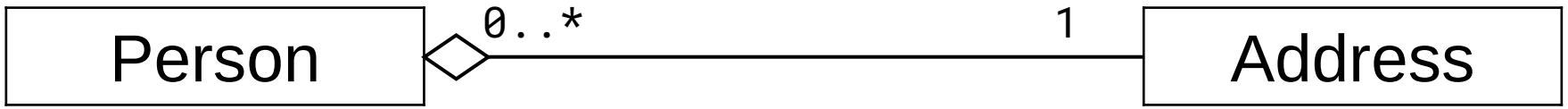
https://www.learncpp.com/cpp-tutorial/aggregation/

Aggregation

- An example to give a sense of aggregation could be,

*A Person **has-a** Address.*

`0..*` *can be replaced by just* `*`



Aggregation

- Notice the following.
  - The part (address) is *(loosely)* part of the object (Person).
  - The part (address) may belong to more than one object (Person) at a time.

- Way to think about this problem is that **`Addresses`** already exist in the world regardless of whether **`Persons`** reside there or not.

- **`Persons`** also exist with or without an **`Address`**.

This being a pointer has two consequences which we'll discuss later.

```
class Person
{
public:
    Person(int age, const std::string& name, Address* home = nullptr)
        : m_age{ age }
        , m_name{ name }
        , m_homeAddress{ home }
    {}

    Person(const Person& per) = delete;
    Person& operator= (const Person& per) = delete;

private:
    int m_age;
    std::string m_name;
    Address* m_homeAddress;
};
```

```
class Address
{
public:
    Address(char category, Point3D loc)
        : m_category{ category }, m_location{ loc }
    {}

    // Two addresses cannot occupy the same location on
the map.
    Address(const Address& addr) = delete;
    Address& operator= (const Address& addr) = delete;

private:
    char m_category;   // resid., comm., industr
    Point3D m_location;
};
```

- The part (**`Address`**) is *(loosely)* part of the object (**`Person`**).

- The part (**`Address`**) may belong to more than one object (**`Person`**) at a time.
  - It can be the case that there are more than one **`Persons`** living in the same **`Address`**.
  - In this case, this **lack of independence of data** (or a **shallow copy** of sorts) is actually part of the problem setup!

*Sharing of an **`Address`** is made possible through this attribute being a pointer instead of an object.*

```
class Person
{
public:
    Person(int age, const std::string& name, Address* home = nullptr)
        : m_age{ age }
        , m_name{ name }
        , m_homeAddress{ home }
    {}

    Person(const Person& per) = delete;
    Person& operator= (const Person& per) = delete;

private:
    int m_age;
    std::string m_name;
    Address* m_homeAddress;
};
```

```
class Address
{
public:
    Address(char category, Point3D loc)
        : m_category{ category }, m_location{ loc }
    {}

    // Two addresses cannot occupy the same location on
the map.
    Address(const Address& addr) = delete;
    Address& operator= (const Address& addr) = delete;

private:
    char m_category;   // resid., comm., industr
    Point3D m_location;
};
```

Aggregation

- Notice the following.
  - The part (address) is *(loosely)* part of the object (Person).
  - The part (address) may belong to more than one object (Person) at a time.
  - The part (address) does not have its existence managed by the object (Person).

- The part (**Address**) does not have its existence managed by the object (**Person**).

- As mentioned earlier, an **Address** can be in existence, even if no **Person** has it as their home.
  - Therefore, the lifetime of an **Address** is not dependent on the lifetime of a **Person** (unlike composition).
    - The creation (or birth) of a **Person** does not result in the creation of an **Address**.
    - The destruction (or death) of a **Person** does not result in the destruction of an **Address**.

```
class Person
{
public:
    Person(int age, const std::string& name, Address* home = nullptr)
        : m_age{ age }
        , m_name{ name }
        , m_homeAddress{ home }
    {}

    Person(const Person& per) = delete;
    Person& operator= (const Person& per) = delete;

private:
    int m_age;
    std::string m_name;
    Address* m_homeAddress;
};
```

```
class Address
{
public:
    Address(char category, Point3D loc)
        : m_category{ category }, m_location{ loc }
    {}

    // Two addresses cannot occupy the same location on
the map.
    Address(const Address& addr) = delete;
    Address& operator= (const Address& addr) = delete;

private:
    char m_category;   // resid., comm., industr
    Point3D m_location;
};
```

- This is the **second consequence** of having a pointer to **`Address`** (**`Address*`**).

- Had this been just an **`Address`** object, the lifetime of the **`Address`** would be dependent on the lifetime of the **`Person`**.

```
class Person
{
public:
    Person(int age, const std::string& name, Address* home = nullptr)
        : m_age{ age }
        , m_name{ name }
        , m_homeAddress{ home }
    {}

    Person(const Person& per) = delete;
    Person& operator= (const Person& per) = delete;

private:
    int m_age;
    std::string m_name;
    Address* m_homeAddress;
};
```

```
class Address
{
public:
    Address(char category, Point3D loc)
        : m_category{ category }, m_location{ loc }
    {}

    // Two addresses cannot occupy the same location on
the map.
    Address(const Address& addr) = delete;
    Address& operator= (const Address& addr) = delete;

private:
    char m_category;   // resid., comm., industr
    Point3D m_location;
};
```

Aggregation

- Notice the following.
  - The part (address) is *(loosely)* part of the object (Person).
  - The part (address) may belong to more than one object (Person) at a time.
  - The part (address) does not have its existence managed by the object (Person).
  - The part (address) does not know about the existence of the object (Person).

- The part (**`Address`**) does not know about the existence of the object (**`Person`**).

- This is a **unilateral relationship** – the **`Person`** knows about their home (through the **`Address*`**) but the **`Address`** has no idea about the **`Persons`** residing in it.

```
class Person
{
public:
    Person(int age, const std::string& name, Address* home = nullptr)
        : m_age{ age }
        , m_name{ name }
        , m_homeAddress{ home }
    {}

    Person(const Person& per) = delete;
    Person& operator= (const Person& per) = delete;

private:
    int m_age;
    std::string m_name;
    Address* m_homeAddress;
};
```

```
class Address
{
public:
    Address(char category, Point3D loc)
        : m_category{ category }, m_location{ loc }
    {}

    // Two addresses cannot occupy the same location on
the map.
    Address(const Address& addr) = delete;
    Address& operator= (const Address& addr) = delete;

private:
    char m_category;   // resid., comm., industr
    Point3D m_location;
};
```

The part is *(loosely)* part of the object.

- In composition, the part is a **part-of** the object.
  - This is due to all the reasons we saw.

- However, in aggregation, this is a somewhat ***weaker*** relationship due to the reasons we just saw.
  - Lifetime of the part not being tied to the object,
  - multiple objects can share the same part,
  - etc.

- So aggregation is said to model **“has-a”** relationship.

Implementing Aggregations

- There are similarities between implementing aggregations and compositions.
- In compositions, we directly add parts as member variables (or as pointers when composition class handles memory management).
- In aggregations, the member variables are typically pointers or references.
  - These point at parts that have **already** been created.
  - Recall that aggregation class isn’t responsible for managing the lifetime of its parts.

Another example

- Self study.

- Another example is given in the associated reading.

Summarizing composition and aggregation

Compositions:

- Typically use normal member variables
- Can use pointer members if the class handles object allocation/deallocation itself
- Responsible for creation/destruction of parts

Aggregations:

- Typically use pointer or reference members that point to or reference objects that live outside the scope of the aggregate class
- Not responsible for creating/destroying parts

It is worth noting that the concepts of composition and aggregation can be mixed freely within the same class. It is entirely possible to write a class that is responsible for the creation/destruction of some parts but not others. For example, our Department class could have a name and a Teacher. The name would probably be added to the Department by composition, and would be created and destroyed with the Department. On the other hand, the Teacher would be added to the department by aggregation, and created/destroyed independently.

While aggregations can be extremely useful, they are also potentially more dangerous, because aggregations do not handle deallocation of their parts

You may ignore **`std::reference_wrapper`** *in this associated reading.*

*FYI, there are also other relationships – **association** and **dependency**.*

Composition vs aggregation vs association summary

Here's a summary table to help you remember the difference between composition, aggregation, and association:

Property	Composition	Aggregation	Association
Relationship type	Whole/part	Whole/part	Otherwise unrelated
Members can belong to multiple classes	No	Yes	Yes
Members' existence managed by class	Yes	No	No
Directionality	Unidirectional	Unidirectional	Unidirectional or bidirectional
Relationship verb	Part-of	Has-a	Uses-a

Different reference:
https://www.learncpp.com/cpp-tutorial/association/

(SS) Quiz Time (from the associated reading)

https://www.learncpp.com/cpp-tutorial/aggregation/